Zonary Programming Language Specification

Version 2.1.1
Made by Kacefier

Github: https://github.com/Kacefier/Zonary
Author Email: kacefier@zohomail.com

I. Introduction

Zonary is a minimalist, binary-based esoteric programming language (esolang) designed for educational and recreational purposes. It is not intended for production use.

The language operates on binary instructions and registers named by binary numbers. All values are unsigned integers within a configurable bit-width.

II. General Syntax

1. Source files use the .zonary extension.

2. The interpreter ignores all spaces, tabs, and newlines.

3. Comments are enclosed in angle brackets: < comment > and may span multiple lines.

4. All instructions and preprocessor directives are composed of binary digits (0 and 1) unless custom characters are defined via preprocessor directives.

III. Bit-Width and Registers

1. Bit-Width

The program bit-width (default: 8) determines the number of bits used for:
- (1) Register names
- (2) Immediate values
- (3) Label numbers

Bit-width can be changed using the /01 preprocessor directive.

All arithmetic results are automatically truncated to 2^bit-width (modulo 2^bit-width).

2. Registers

Registers are named by binary numbers of the current bit-width.

Example (8-bit mode): registers range from 00000000 to 11111111.
Total registers: 2^bit-width.

3. Memory Optimization

Registers are lazily allocated:

(1) A register that has never been assigned a value does not occupy memory.

(2) A register assigned a value of 0 is freed from memory (treated as unassigned).

(3) Reading an unassigned or cleared register returns 0 (no error).

IV. Preprocessor Directives

Preprocessor directives are prefixed with / and are processed Before the main program execution. They are independent of the runtime environment and are not affected by program state.

1. Directive /00

Format: */00* <*char0*> <*char1*>

Description: Replaces default binary characters 0 and 1 with custom ASCII characters. Each parameter is an 8-bit binary ASCII code.

Example: /00 00101110 00101101 uses "." for 0 and "-" for 1.

Notice: Custom characters cannot be <, >, /, 0, 1, or identical to each other.

Fixed length: 18 bits.

2. Directive /01

Format: */01* <*bit-width*>

Description: Sets the program bit-width. Parameter is an 8-bit binary value.

Example: /01 00001000 sets bit-width to 8 (default).

Notice: Must appear before any register or label references that depend on the new width.

Fixed length: 10 bits.

3. Directives /10 and /11

Description: Reserved for future use. No effect in this version.

V. Instruction Set

Each instruction has a fixed binary length and consists of an opcode followed by parameters.

1. Opcode 000 - SET

Format: *000* <*register*> <*parameter*> <*mode*>
Length: *4* + *bit-width* * *2* bits

Description: Assigns a value to the specified register.

Mode (1 bit):
- 0: Parameter is an immediate value; assign directly.
- 1: Parameter is a register number; copy value from that register.

2. Opcode 001 - ADD

Format: *001* <*registerA*> <*parameter*> <*mode*>
Length: *4* + *bit-width* * *2* bits

Description: Adds a value to registerA, stores result in registerA.

Mode (1 bit):
- 0: Add immediate value.
- 1: Add value from register.

Notice: Result is automatically truncated to 2^bit-width.

3. Opcode 010 - SUB

Format: *010* <*registerA*> <*parameter*> <*mode*>
Length: *4* + *bit-width* * *2* bits

Description: Subtracts a value from registerA, stores result in registerA.

Mode (1 bit):
- 0: Subtract immediate value.
- 1: Subtract value from register.

Notice: Result is automatically truncated to 2^bit-width.

4. Opcode 011 - JMP

Format: *011* <*parameter*> <*mode1*> <*mode2*>
Length: *5* + *bit-width* bits

Description: Unconditionally jumps to a specified label, or defines a new label.

mode1 (1 bit):
    0: Jump to label.
    1: Define label (points to next instruction).

mode2 (1 bit):
    0: Parameter is immediate label number.
    1: Parameter is register containing label number.

Notice: If the target label does not exist, a warning is issued and the jump is ignored; execution continues with the next instruction. If a label number is redefined, an error is reported and the program terminates.

5. Opcode 100 - IFZ

Format: *100* <*register*> <*parameter*> <*mode*>
Length: *4* + *bit-width* * *2* bits

Description: If the specified register value is 0, jumps to the specified label.

Mode (1 bit):
  0: Parameter is immediate label number.
  1: Parameter is register containing label number.

Notice: If the register value is not 0, execution continues with the next instruction. If the label does not exist, a warning is issued and the jump is ignored.

6. Opcode 101 - OUT

Format: *101* <*parameter*> <*mode1*> <*mode2*>
Length: *6* + *bit-width* bits

Description: Outputs a value. Does not automatically append a newline.

mode1 (2 bits) - Output format:
    00: Binary (fixed bit-width digits).
    01: Decimal.
    10: Hexadecimal (uppercase, fixed to bit-width/4 digits).

11: ASCII character (displays character if value is 32~126, otherwise shows \xXX escape sequence).

mode2 (1 bit):
0: Parameter is immediate value.
1: Parameter is register; output its value.

7. Opcode 110 - INP

Format: *110* <*register*> <*mode*>
Length: *5* + *bit-width* bits

Description: Reads data from standard input and stores it in the specified register. Leading whitespace (spaces, newlines, etc.) is skipped.

Mode (2 bits) - Input format:
00: Binary (fixed bit-width digits).
01: Decimal (0 to 2^bit-width - 1).
10: Hexadecimal (case-insensitive, fixed to bit-width/4 digits).
11: Single ASCII character; stores its ASCII code.

Notice: If input is malformed or empty, the register is set to 0.

8. Opcode 111 - SYS

Format: *111* <*parameter*> <*mode*>
Length: *4* + *bit-width* bits

Description: Performs a system function.

Mode (1 bit):
0: Terminate program; output the parameter value as the return code in binary.
1: Delay execution; parameter specifies milliseconds to delay (0 to 2^bit-width - 1 ms).

Notice: If the program runs through all instructions without encountering SYS 0, it terminates with a default return code of all zeros (0).

VI. Execution Model

1. Programs execute sequentially from the first instruction.

2. Jumps and labels are resolved at runtime.

3. Preprocessor directives are processed before execution.

4. If execution reaches the end of the program without a SYS 0, it terminates with return code 0.

VII. Error and Warning Handling

1. Error

Triggered when a fatal problem is encountered during parsing or execution. The program terminates immediately and outputs an error message. Includes but is not limited to:

(1) Preprocessor directive parameter errors (e.g., invalid /00 parameters, /01 bit-width less than 2).
(2) Binary code format errors (e.g., incomplete instructions, unknown opcodes, invalid characters).
(3) Duplicate label definitions.

2. Warning

Triggered when a non-fatal problem is encountered. A warning message is output but execution continues. Includes but is not limited to:

(1) JMP or IFZ jumping to a non-existent label.
(2) Extra trailing bits in the code (ignored).

3. Silent Fallback

When an exception is encountered, the program automatically applies default handling without outputting any information. Includes but is not limited to:

(1) When reading an unassigned or cleared register, the value 0 is returned.
(2) When INP input is malformed or empty, the register is set to 0.

VIII. Etymology

The name Zonary is derived from:
Zero + **One** + **Binary** = **Zonary**

- The End -